

Load Balancing Algorithms

Prepared By: Dr. D. P. Singh

Load Balancing Algorithms

- Round Robin
- Least Connections
- Least Response time
- IP Hash
- Weighted Algorithms
- Geographical Algorithms
- Consistent Hashing

Round Robin

- Round-robin load balancing is a, straightforward, and highly popular algorithm used to **distribute incoming network traffic across a group of backend servers**.
- **It functions on a rotational basis**: the load balancer assigns the first request to the first server in the list, the second request to the next server, and continues down the line before starting over at the top.
- This method is best suited for environments with **identical, equally powerful servers** handling similar, predictable, and stateless requests.

Round Robin

contd...

Advantages:

- **Simplicity:** Very easy to implement and understand.
- **Fairness:** Distributes requests evenly across all servers, ensuring no single server is overburdened.
- **Low Overhead:** Requires minimal computational power on the load balancer itself.

Round Robin

contd...

Disadvantages:

- **Ignores Server Health/Load:** It assumes all servers have the **same capacity and are always healthy**. A slow server or one with long-lived connections can cause a bottleneck.
- **Not Ideal for Heterogeneous Servers:** If one server is more powerful than others, a standard round-robin **will not utilize its extra capacity**, leading to inefficiency.
- **Not Suitable for Stateful Sessions:** Without "sticky sessions" (persistence), a user might be routed to a **different server for their next request, potentially losing session data**.

Least Connections

- The load balancer keeps track of the active connections for each server in the pool. When a new request arrives, it **selects the server currently handling the smallest number of active, concurrent requests.**
- It works by monitoring real-time load, often defaulting to round-robin if connections are tied.

Least Connections

contd...

Advantages:

- **Efficient Load Distribution:** Prevents a single server from becoming overwhelmed while others are idle.
- **Improved Performance:** Enhances user experience by directing traffic to the least busy server.

Disadvantages:

- **Complexity:** More difficult to implement than simple algorithms like round-robin, as it requires constant tracking of connection states.
- **Polling Overhead:** Requires frequent updates, which can add overhead to the load balancer itself.

Least Response Time

- Least Response Time (LRT) load balancing is a **dynamic algorithm that directs incoming traffic to the server with the lowest active connections and the fastest response time.** often used for HTTP/HTTPS traffic
- **Performance Tracking:** The load balancer continuously monitors the latency (response time) of each server, including the time taken to process requests and return the first byte.
- **Selection Criteria:** The algorithm **selects the best-performing node using a combination of the lowest current latency and the fewest active connections.**
- **Error Handling:** If a server returns an error, it is often treated as having a high response time (long latency), ensuring it is bypassed in favor of healthier nodes.
- **Dynamic Adjustment:** Because it calculates metrics in real-time, the **algorithm adapts to fluctuating server performance and network traffic.**

Least Response Time

contd...

Advantages:

- **Optimized User Experience:** Ensures users are routed to the fastest responding server, reducing latency.
- **Resource Intensity:** Requires more computational overhead than simpler algorithms like Round Robin, as it necessitates constant monitoring of server performance.
- **Best Use Cases:** Ideal for scenarios where server response times vary significantly, such as diverse backend services or cloud-native applications

Least Response Time

contd...

Disadvantages:

- **High Monitoring Overhead:** Requires constant, real-time tracking of each server's health and latency, which demands significant resources.
- **Complexity:** Implementation is more complex than static methods like Round Robin.
- **Misleading Metrics:** A server might have a low response time, yet be under heavy load, leading to poor decisions.
- **False Positives on Errors:** A malfunctioning server returning quick 404 error codes might be favored over a healthy server that is slower.
- **Oscillation Risk:** Rapidly changing metrics can cause the algorithm to switch traffic between servers constantly, leading to instability.
- **No Session Persistence:** It does not inherently support session stickiness, which is problematic for stateful applications

- Thank You